Attention-Enhanced Prioritized Proximal Policy Optimization for Adaptive Edge Caching

Farnaz Niknia, Ping Wang, Zixu Wang, Aakash Agarwal, and Adib S. Rezaei

***Abstract*—This paper tackles the growing issue of excessive data transmission in networks. With increasing traffic, backhaul links and core networks are under significant traffic, leading to the investigation of caching solutions at edge routers. Many existing studies utilize Markov Decision Processes (MDP) to tackle caching problems, often assuming decision points at fixed intervals; however, real-world environments are characterized by random request arrivals. Additionally, critical file attributes such as lifetime, size, and importance significantly impact the effectiveness of caching policies, yet existing research fails to integrate all these attributes in policy design. In this work, we model the caching problem using a Semi-Markov Decision Process (SMDP) to capture the continuous-time nature of real-world applications better, enabling caching decisions to be triggered by random file requests. We then introduce a Proximal Policy Optimization (PPO)-based caching strategy that fully considers file attributes like lifetime, size, and priority. Simulations show that our method outperforms a recent Deep Reinforcement Learning-based technique. To further advance our research, we improved the convergence rate of PPO by prioritizing transitions within the replay buffer through an attention mechanism. This mechanism evaluates the similarity between the current state and all stored transitions, assigning higher priorities to transitions that exhibit greater similarity.**

***Index Terms*—Attention mechanism, caching, proximal policy optimization, semi-Markov decision process.**

I. Introduction

With the rapid expansion of mobile applications, there has been a notable rise in redundant data transmissions due to numerous users requesting content from centralized data centers. This surge in traffic has significantly strained backhaul links and core networks [1]. Consequently, edge caching at routers has emerged as a promising strategy to alleviate traffic redundancy and reduce transmission delays, as demonstrated by recent research [2], [3].

Received 27 October 2024; revised 31 January 2025; accepted 19 February 2025. Date of publication 2 April 2025; date of current version 15 August 2025. This work was supported by a Canada NSERC Discovery grant. The review of this article was coordinated by Dr. Hongyang Du. *(Corresponding author: Ping Wang.)*

Farnaz Niknia and Ping Wang are with the Lassonde School of Engineering, York University, Toronto, ON M3J 1P3, Canada (e-mail: fniknia@yorku.ca; pingw@yorku.ca).

Zixu Wang is with the School of Electronic and Computer Engineering, The Hong Kong University of Science and Technology, Hong Kong 999077, China (e-mail: zwangjx@connect.ust.hk).

Aakash Agarwal is with the School of Engineering and Applied Science, University of Pennsylvania, Philadelphia, PA 19104 USA (e-mail: aakash24@seas.upenn.edu).

Adib S. Rezaei is with the Department of Computer Science, Virginia Tech, Blacksburg, VA 24061 USA (e-mail: adibzi@vt.edu).

Digital Object Identifier 10.1109/TVT.2025.3552084

Current caching strategies can be classified into two main types: reactive and proactive [4], [5]. Reactive caching entails deciding whether to store a file only after it has been requested [1], [6], [7]. Proactive methods, in contrast, rely on historical data to forecast future file popularity and pre-cache the content even before any requests are made [8], [9], [10]. However, a key limitation of proactive caching is that it may result in low cache-hit ratios if the predicted content fails to become popular. Additionally, pre-caching content that may never be accessed wastes communication bandwidth and valuable storage space, making reactive methods a more efficient alternative.

Another essential consideration is that caching inherently involves sequential decision-making, effectively modeled by the Markov Decision Process (MDP) framework, as outlined in [11]. An MDP is defined by a set of states and actions available to an agent, who receives rewards based on their actions and the resulting states. The agent's goal is to derive a policy that maximizes long-term expected rewards. If the model of the environment, such as the reward function and transition probabilities, is known, MDPs can be solved using Bellman optimality equations [12]. However, in many practical scenarios, such information is unavailable. In these cases, model-free Reinforcement Learning (RL) techniques are employed, allowing an agent to learn from the environment through exploration and trial-and-error rather than relying on pre-existing knowledge of the environment [12].

Most existing studies apply the MDP framework to model caching problems [13]. However, MDP-based approaches assume decisions are made at fixed intervals [14], which is unsuitable for scenarios where caching decisions need to be made upon the arrival of requests. MDP is often inadequate since real-world environments feature requests arriving randomly at the edge router [15] MDP is often inadequate. A more fitting alternative is the Semi-Markov Decision Process (SMDP) [11], [12], which accommodates state transitions occurring at uneven intervals. Like MDP, SMDP provides a framework for modeling decision-making processes, but it offers more flexibility since state transitions can occur at equal, exponential, or random intervals [11]. MDP can be seen as a special case of SMDP where the intervals between state transitions are one unit of time. The work proposed in [16] exemplifies the application of SMDP for caching.

Another critical aspect of the caching problem is that not all files have the same level of importance. Importance here reflects how much a user values having a file stored nearby to avoid delays in future access. For instance, some files may hold

0018-9545 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

critical importance for users, such as those required for urgent tasks or containing high-value information. If the caching system prioritizes storing these highly important files, it can deliver greater benefits by ensuring quick and reliable access to the most needed resources. Files with different levels of importance often directly influence user satisfaction or operational efficiency, making the prioritization of crucial files an essential strategy for maximizing the system's effectiveness. This approach provides a significant advantage to the caching system: prioritizing files based on their importance to users can enhance its overall benefit and efficiency. In addition to the importance of files, other attributes like size and lifetime also play a key role in optimizing caching policies. For instance, consider a scenario where a popular file is large but has a short lifetime. Caching this file might require evicting several other files from the cache. If the file expires before it is reaccessed, this would not only make caching ineffective but could also degrade overall system performance. Yet, most existing caching policies fail to account for all these attributes when making decisions.

Just as optimizing caching decisions requires careful consideration of these file characteristics, optimizing reinforcement learning algorithms also depends on making smart choices about which experiences to prioritize during training. Prioritizing transitions closely resembling the current system state accelerates convergence by ensuring that the agent focuses on the most relevant experiences, directly informing the best policy updates. By concentrating on transitions closely related to its current state, the agent learns from experiences more applicable to its present situation, increasing the likelihood of policy effectiveness. This targeted approach reduces time spent on less relevant experiences, leading to more efficient learning and fewer updates needed for policy improvement. Moreover, learning from similar states minimizes variance in policy updates, stabilizing the learning process and resulting in smoother and more consistent progress toward convergence.

Motivated by these challenges, this paper proposes a reactive caching method built upon the SMDP framework [11], allowing for decision-making at random intervals, particularly when a file is requested at the edge router. Our main contributions are as follows:

- We model the caching problem using SMDP, which better reflects the real-time nature of request arrivals. We also present a PPO-based caching strategy that leverages historical popularity data to develop a caching policy while accounting for the system's inherent uncertainties.
- We incorporate multiple file attributes such as lifetime, size, importance, and popularity in our caching decisions. To the best of our knowledge, our method is the first to integrate all these attributes into a comprehensive caching strategy, making it more applicable to practical environments.
- Through simulations, we assess our method's performance under various scenarios and compare it against two recent Deep Reinforcement Learning (DRL)-based approaches that consider both file popularity and lifetime [17] and [6]. Results show that our approach consistently achieves a higher cache hit rate and total utility across different configurations, including varying cache sizes, request rates, and popularity distributions.
- We improve the convergence speed of the PPO algorithm by incorporating an attention mechanism to prioritize transitions in the replay buffer. This mechanism evaluates the similarity between the current state and all transitions in the replay buffer, assigning higher priority to those with greater similarity. This approach accelerates convergence by focusing learning on more relevant transitions.

The remainder of the paper is structured as follows: Section II provides an overview of related work. Section III introduces our system model, while Section IV formulates the caching problem. Section V outlines our proposed caching algorithm. Section VI discusses we discuss our experimental setup and results, and Section VII concludes the paper.

II. Related Work

A. Reactive Caching

The study in [7] introduces a new metric called 'virality', which measures the variation in file popularity over time. The authors use this metric along with popularity and size, to prioritize which files should be cached. Another approach detailed in [1] involves a recommendation system-based model to predict the popularity of newly requested content. This prediction guides a DRL-based caching strategy to optimize caching decisions by balancing latency and request frequency.

In [17], the authors address the trade-off between communication costs and data freshness using an actor-critic DRL approach. They propose a utility function that combines these factors to improve caching efficiency. Similarly, in [6] the authors employ the PPO algorithm to enhance cache hit rates while minimizing energy use. A variant of this approach is presented in [18], where the DRL agent is penalized based on the age of cached files and available cache memory.

In [19], the issue of average data transmission delay within cache storage constraints is tackled using deep reinforcement learning, initially formulated as an Integer Linear Programming (ILP) problem before applying DRL. The authors of [20] propose deep actor-critic methods for reactive caching, focusing on maximizing cache hit rates and managing transmission delays in centralized and decentralized settings.

Study [21] integrates recommender systems with edge caching in mobile edge-cloud networks, aiming to reduce the long-term system cost by modeling user experience factors. In [22], user preferences influence cache management, with higher preference content replacing lower preference items when space is limited. The study in [23] focuses on vehicular networks, using region-based models to optimize content fetching locations and employing the Least Recently Used (LRU) strategy for cache management.

B. Proactive Caching

Study [24] addresses the 'slow start' problem in caching algorithms by calculating Euclidean distances to identify file similarities, assuming similar files are likely to be popular. In [8],

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

the caching of multi-view 3D videos is modeled models the caching of multi-view 3D videos using an MDP, combining Deep Deterministic Policy Gradient (DDPG) with a dynamic k-nearest neighbor (KNN) algorithm to adapt to varying action spaces.

In [9], the device-to-device (D2D) caching problem is formulated as a multi-agent multi-armed bandit problem, with Q-learning used to coordinate user caching decisions. A modified combinatorial Upper Confidence Bound (UCB) algorithm is employed to manage extensive action spaces. Study [10] focuses on optimizing caching placement in a two-tier system, using Difference of Convex (DC) programming to maximize offloading probability through local caching and sharing.

Study [25] proposes a computation offloading method that combines demand prediction using a Spatial-Temporal Graph Neural Network (STGNN) with a caching decision algorithm based on predicted demand. The authors in [26] address varying content popularity by predicting average popularity and adjusting caching probabilities accordingly, managing different content popularities across runtime sessions.

Proactive caching faces challenges, such as potential low cache hit rates for pre-cached unpopular content and limited adaptability to dynamic user behavior changes. If user preferences shift and previous caching decisions no longer apply, this can result in suboptimal performance.

C. Motivation

Real-world systems often contain files with diverse attributes that impact caching effectiveness, including lifetime, importance, and size. Our review reveals that current methods do not fully consider these attributes in caching decisions. Our proposed policy is the first to integrate a comprehensive set of file characteristics, offering a novel approach to caching.

Additionally, many existing methods model caching as a discrete-time problem [1], [6], [8], [9], [13], [17], [19], [20], [27], which conflicts with the continuous-time nature of request arrivals. We address this by formulating the problem as an SMDP and developing a DRL-based reactive caching scheme. This approach maintains a brief request history and employs PPO to optimize the DRL agent's policy, enabling it to make adaptive caching decisions based on current and past experiences.

III. System Model

A. System Architecture

We consider a network scenario where end users access the Internet through an edge router, which connects to a data center on one side and the end users on the other, as illustrated in Fig. 1.

The cloud data center is assumed to have sufficient capacity to store all contents [24]. When an end user requests a file, a copy is created and sent to the user via the edge router over the Internet. The edge router has a limited cache capacity. If the requested file f_r is already cached, the request is fulfilled from the edge router's cache rather than retrieving it from the data center. Hereafter, we use "cache" to refer to the edge router's cache. End users are devices that request files based on their needs and preferences [24]. Let $\mathcal{U} = \{u_1, u_2, \ldots, u_u\}$ denote the users connected to the edge router. Requests are represented by $\mathcal{G} = \{g_1, g_2, \ldots, g_g\}$, where g_g denotes the gth request, irrespective of the user making it. Requests are processed in the order they are created.

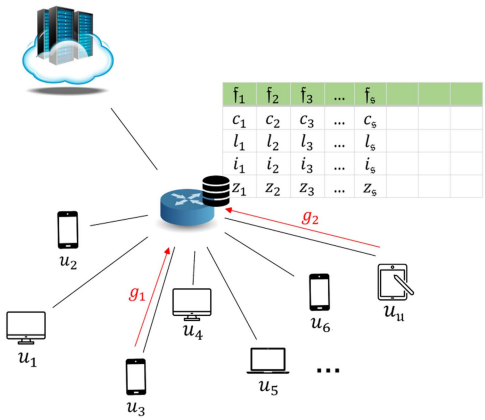


Fig. 1. Caching system topology. Users u_1, u_2, u_3, and u_u are connected to the router. Users u_3 and u_u send requests g_1 and g_2 respectively. File types f_1, f_2, f_3 and f_s are already cached in the edge router. c_1, l_1, i_1 and z_1 are request history, life time, importance and size of f_1.

Files, generated by cameras, sensors, and computers, are stored in the data center. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_f, \ldots, f_F\}$ denote the set of file types, where f_f represents the fth type of file. Each file type may have distinct characteristics including popularity, lifetime, size, and importance. These characteristics are further explained with real-life examples:

- *Popularity:* The number of times a file is requested by users [28]. Popular files, like trending videos, are requested more frequently than less popular ones.
- *Lifetime:* The duration for which a file is valid from when it leaves the data center [18]. For example, location-based services require timely updates, so files have a predetermined lifetime upon generation [29].
- *Size:* Files may vary in size depending on their type and content. For instance, a movie file is typically larger than a text file.
- *Importance:* Reflects a file's value for users based on its relevance and necessity. Files crucial for timely access, such as real-time financial data or emergency updates, are considered more important than less critical files like free ebooks or casual videos.

A caching policy must consider all file characteristics simultaneously. Focusing on only one attribute may overlook other important features. For example, a highly popular file with a short lifetime, large size, and low importance might not be as beneficial as a less popular file with a longer lifetime, smaller size, and higher importance.

In this paper, we consider four characteristics for each file. Let $\mathbf{c} = (c_1, c_2, \ldots, c_f, \ldots, c_F)$ denote the popularity values for each file type, where c_f represents the popularity of file type f. This indicates the number of times file type f is requested within a predefined time interval $\mathcal{T}$. We denote the lifetime, size, and

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

importance of each file type by $\mathbf{l} = (l_1, l_2, \ldots, l_f, \ldots, l_F)$, $\mathbf{z} = (z_1, z_2, \ldots, z_f, \ldots, z_F)$, and $\mathbf{i} = (i_1, i_2, \ldots, i_f, \ldots, i_F)$ respectively.

The utility of a cached file, defined as a function of its freshness and importance, is used to determine its value. Freshness is the age of the file normalized by its lifetime at time t: $h^f(t) = (t - w_g^f)/l_f, \quad 0 \leq h^f(t) \leq 1$, where t is the current time, and w_g^f is the generation time of file type f.

The utility function for file type f at time t is denoted as:

$$y_f(t) = E(h^f(t), i_f), \tag{1}$$

where $y_f(t)$ represents the utility of file type f at time t. The function E depends on two key factors: the freshness of the file, denoted as $h^f(t)$, and the importance of the file type, represented by i_f. The function E captures the interaction between these factors in determining the overall utility. In general, the utility $y_f(t)$ increases as the importance i_f of the file grows, reflecting the higher benefit the file brings to the system. Notably, there is an inverse relationship between the file's freshness and its utility; as the freshness $h^f(t)$ decreases over time, indicating the file is becoming older, the utility $y_f(t)$ correspondingly. While the definition of a utility function often varies according to individual user perspectives and specific applications, there is no universally applicable definition. However, our proposed method is flexible and can be utilized with any definition of a utility function.

B. System Uncertainties

Caching involves uncertainties such as random request arrivals and the unpredictable impact of future requests on the cache. These uncertainties impact the decision-making process of the edge router. Below, we outline these uncertainties and our assumptions.

1) Random Arrivals of Requests: Requests for content often arrive randomly due to factors like user behavior and network conditions. We model request arrivals using a Poisson process, a common model for characterizing user request patterns [30]. The parameter λ represents the request rate, and μ is the expected time between consecutive requests, where $\lambda = 1/\mu$. We assume the edge router has no prior knowledge of the Poisson process or its parameters.

2) Unknown Effect of Upcoming Requests on the Cache: Given the F types of files with unique characteristics (popularity, lifetime, size, importance), there is no prior knowledge of future content requests. Consequently, caching a file may have an unforeseen impact on subsequent decisions.

These uncertainties significantly affect caching performance. In Section V, we discuss how our caching policy addresses these uncertainties and propose an approach to optimize the hit rate in the long run.

IV. Problem Formulation

The edge router's task involves deciding which files to cache based on several factors, including the frequency of user requests, file characteristics, and the available cache space. This sequential decision-making process can be effectively modeled using MDPs, as detailed in [11].

An MDP comprises five components: state, action, system dynamics, reward function, and policy. In this framework, a state represents the system's status at a given time. The agent selects an action based on the policy, transitions to a new state, and receives a reward reflecting the action's quality. The process continues indefinitely (infinite horizon) or until a final state is reached (finite horizon) [12]. For the caching problem, the infinite-horizon case is appropriate since there is no final state.

Given that request arrivals are continuous, a continuous-time variant of MDP is necessary. We propose using the SMDP framework, which accommodates variable transition times [11]. The following sections elaborate on the SMDP formalism and its components.

A. SMDP Formalism

An SMDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, J, R, \pi)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, J represents transition times, R is the reward function, and π denotes the policy.

1) States of the System: The system state at time t is denoted by $s_t \in \mathcal{S}$ and includes: $s(t) = \{Mem(t), \mathbf{b}(t), \mathbf{y}(t), \mathbf{d}(t), \mathbf{i}(t), \mathbf{l}(t), \mathbf{z}(t)\}$, where $Mem(t)$ indicates the unoccupied percentage of cache memory, expressed as $Mem(t) = (M - \sum_{f=1}^{F} \mathbf{b}(t) \cdot \mathbf{z}(t))/M$, where M is the cache capacity. The vector $\mathbf{b}(t) = (b_1(t), b_2(t), b_3(t), \ldots, b_f(t), \ldots, b_F(t))$, $\mathbf{b}(t) \in B = \{0, 1\}^F$, is a binary vector, where a value of 0 indicates that a file is not cached and a value of 1 indicates that the file is already cached. The vectors $\mathbf{y}(t) = (y_1(t), y_2(t), y_3(t), \ldots, y_f(t), \ldots, y_F(t))$, indicates the utility of files and $y_f(t) = 0$ if f is not currently cached. $\mathbf{d}(t) = (d_1(t), d_2(t), d_3(t), \ldots, d_f(t), \ldots, d_F(t))$ represents the number of times that each file type has been requested within recent N requests. $\mathbf{i}(t) = (i_1(t), i_2(t), i_3(t), \ldots, i_f(t), \ldots, i_F(t))$, $\mathbf{l}(t) = (l_1(t), l_2(t), l_3(t), \ldots, l_f(t), \ldots, l_F(t))$ and $\mathbf{z}(t) = (z_1(t), z_2(t), z_3(t), \ldots, z_f(t), \ldots, z_F(t))$ represent the importance, lifetime and size of each file type within recent N requests.

2) Actions: At any time t, the agent can take one of two actions: $a(t) = 1$ (cache the file) or $a(t) = 0$ (do not cache the file). When the cache is full, the file with the lowest utility is removed to make space for the new file. If more space is needed, files with the next lowest utilities are removed.

3) Dynamics of the System: In our system, formulated as an SMDP with random task arrivals, the dynamics are governed by both the state transition probabilities $(P_{ss'})$ and the transition times between states (τ). This makes the system's behavior more complex than standard MDPs, as time and state transitions influence decision-making. If these dynamics along with the reward function, were fully known, Bellman equations could be used to obtain the optimal policy. However, since such information is often unavailable in real-world problems, we employ reinforcement learning to learn these dynamics and optimize decision-making through experience iteratively. The specifics of the RL algorithm applied will be described in the next section.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

4) Instant Reward and Long-Term Goal: The instant reward is defined as:

$$r(t) = w_1((\mathbf{b}(t) \cdot \mathbf{d}(t))(\mathbf{b}(t) \cdot \mathbf{y}(t))^T) - w_2 Mem(t), \quad (2)$$

where the first term represents the weighted utility of cached files, and the second term accounts for the unused cache space. The coefficients w_1 and w_2 are weighting factors that balance the trade-off between prioritizing high-utility cached files and efficient cache utilization.

The first term of the reward function, $w_1((\mathbf{b}(t) \cdot \mathbf{d}(t))(\mathbf{b}(t) \cdot \mathbf{y}(t))^T)$, captures the combined effect of file demand and utility. The dot product $\mathbf{b}(t) \cdot \mathbf{d}(t)$ computes the total demand for files currently stored in the cache, ensuring that only cached files contribute to this value. This is then weighted by $\mathbf{b}(t) \cdot \mathbf{y}(t)$, which represents the aggregated utility of cached files. By prioritizing files that are both highly requested and highly valuable, this term incentivizes the caching system to align its strategy with user preferences and maximize the system's benefit.

The second term, $-w_2 Mem(t)$, penalizes unutilized cache space to motivate the system to make full use of the cache. By filling the cache with high-demand and high-utility files, the system ensures efficient utilization of available resources while avoiding underutilization.

The long-term goal of the caching system is to maximize the average accumulated worth of cached files while minimizing the average unoccupied portion of the cache. Using the Proximal Policy Optimization (PPO) algorithm, the caching system learns a policy that dynamically adapts to changing file request patterns and utilities. By optimizing this reward function, the system ensures that it prioritizes caching strategies that yield the highest overall benefit in both the short and long term. This approach reflects the practical considerations of real-world caching scenarios, where demand, utility, and memory constraints must all be carefully managed.

5) The Policy: The policy (π) determines the optimal action for each state to achieve the long-term goal.

V. Enhanced PPO Algorithm With Prioritized Replay Buffer Using Attention Mechanisms

We chose PPO over other reinforcement learning algorithms, such as Deep Deterministic Policy Gradient (DDPG) and Double Deep Q-Learning (DDQL), because of its suitability for the edge caching problem. DDPG is designed for problems with continuous action spaces, while the actions in the edge caching problem are discrete. This mismatch makes DDPG less effective for this application, as it requires unnecessary adjustments to handle discrete actions. In contrast, PPO directly supports discrete action spaces, making it a more natural and efficient choice.

Additionally, DDQL is significantly slower than PPO when applied to edge caching. While DDQL provides reliable convergence in many reinforcement learning problems, its computational overhead and slower updates make it less practical for scenarios like edge caching, where timely decisions are critical. PPO, on the other hand, offers faster convergence and maintains stability during training. Its clipped objective function ensures that policy updates remain stable, even in dynamic environments with fluctuating file popularity and user demands.

PPO combines speed and stability, making it the ideal algorithm for the edge caching problem. Its ability to efficiently handle discrete actions and adapt quickly to changes in the environment allows it to outperform other algorithms like DDPG and DDQL in this context. By leveraging these advantages, PPO provides a robust and effective solution for optimizing caching decisions in dynamic and challenging edge computing scenarios.

This section introduces an enhanced version of the PPO [32] algorithm, which incorporates a prioritized replay buffer that leverages attention mechanisms. This enhancement aims to improve learning efficiency and policy performance by prioritizing transitions that are more pertinent to the agent's current state. We first provide a detailed overview of the PPO algorithm, followed by an explanation of how the transitions are prioritized using attention mechanisms.

A. Proximal Policy Optimization

PPO is a popular reinforcement learning algorithm known for its stability and ease of implementation. PPO addresses the challenges of policy optimization by introducing a clipped objective function, which prevents large policy updates and stabilizes training.

The core idea of PPO is to maximize the expected reward while ensuring that the new policy does not deviate excessively from the old policy. PPO achieves this through a surrogate objective function, which is given by:

$$L^{\text{PPO}}(\theta) = \min\left(\frac{\pi_\theta(a(t) \mid s(t))}{\pi_{\theta_{\text{old}}}(a(t) \mid s(t))}\hat{A}(t), \right.$$
$$\left. \text{clip}\left(\frac{\pi_\theta(a(t) \mid s(t))}{\pi_{\theta_{\text{old}}}(a(t) \mid s(t))}, 1-\epsilon, 1+\epsilon\right)\hat{A}(t)\right), \quad (3)$$

where $\pi_\theta(a(t)|s(t))$ is the probability of taking action a_t in state s_t under the policy parameterized by θ, $\pi_{\theta_{\text{old}}}(a(t)|s(t))$ is the probability under the previous policy, $\hat{A}(t)$ is the advantage function, and ϵ is a clipping parameter.

PPO updates the policy by maximizing this objective function using stochastic gradient ascent. The clipped objective ensures that the new policy does not deviate significantly from the old policy, balancing exploration and exploitation.

The advantage function $\hat{A}(t)$ measures the relative value of an action compared to the baseline:

$$\hat{A}(t) = r(t) + \gamma^\tau V_{\theta'}(s') - V_{\theta'}(s), \quad (4)$$

where $V_{\theta'}(s)$ and $V_{\theta'}(s')$ represent the values of the current state and the next state, respectively. In the context of a standard MDP, τ is equal to 1 because state transitions occur at regular intervals, and the time between consecutive state transitions is fixed. As a result, γ is raised to the power of 1, simplifying the equation. However, in SMDP, the state transition times vary, meaning that τ can take different values depending on the duration between transitions. In this case, the equation is modified by raising γ to the power of τ, which accounts for the variable time intervals between state transitions. This adjustment allows SMDP to more

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

accurately reflect the delayed rewards over non-uniform time steps, ensuring that the future reward is appropriately discounted based on the actual time elapsed between state transitions [33].

The value function is updated alongside the policy using a separate loss function. The loss for the value function is typically the squared error between the predicted value $V_{\theta'}(s)$ and the target return $R(t)$:

$$L^{\text{value}}(\theta') = \left(V_{\theta'}(s) - R(t)\right)^2. \tag{5}$$

This value loss ensures that an accurate estimation of state values accompanies the policy update.

The target return $R(t)$ is the one-step return defined as:

$$R(t) = r(t) + \gamma^{\tau} V_{\theta'}(s'). \tag{6}$$

This formulation allows the value network to minimize the error between the current estimated value and the calculated return based on the immediate reward and future discounted value.

B. Prioritizing Transitions With Attention Mechanisms

We incorporate a prioritized replay buffer using an attention mechanism to further enhance the PPO algorithm.

1) Attention Mechanism for Prioritization: Our approach utilizes the attention mechanism with the key-query-value (KQV) framework to compute the similarity between the current state and the transitions stored in the replay buffer. Let the current state be denoted as s. and the transitions in the replay buffer be represented as a set $(s_i, a_i, r_i, \tau_i, s_{i+1})$, where each transition in the replay buffer is characterized by its state s_i, action a_i, reward r_i, transition time τ_i and next state s_{i+1}.

We use the following steps to compute the attention-based priority for each transition:

- *Compute Attention Scores:* Define the query Q as embedding the current state s, and let the keys K and values be the embeddings of the states in the replay buffer. The attention score a_i for the transition i is calculated using:

$$a_i = \frac{\exp(\text{score}(Q, K_i))}{\sum_{k=1}^{|\Lambda|} \exp(\text{score}(Q, K_k))}, \tag{7}$$

  where $|\Lambda|$ is the size of the replay buffer and exp refers to exponential function. The score function $\text{score}(Q, K_i)$ measures the similarity between the query Q and the key K_i. We use the dot product as the score function as follows:

$$\text{score}(Q, K_i) = Q^T \cdot K_i.$$

- *Calculate Priorities:* The priority p_i of a transition is proportional to its attention score. Define the priority as:

$$p_i = a_i,$$

- *Update Probabilities:* The probability of transition i being sampled is:

$$P(i) = \frac{p_i^{\alpha}}{\sum_{e \in \Lambda} p_e^{\alpha}},$$

  where the value of α determines the degree of prioritization, with $\alpha = 0$ corresponding to the uniform sampling case, i.e., transition i is sampled randomly. To adapt to environment change, adjustments are made by applying importance sampling weights, as illustrated below:

$$\omega_i = \left(\frac{1}{|\Lambda|} \cdot \frac{1}{P(i)}\right)^{\beta},$$

  where β controls the degree of importance sampling. When β is 0, there is no importance sampling, whereas when β is 1, full importance sampling is employed. ω_i is multiplied by the loss function to control the impact of each transition on updating the neural network.

By integrating this attention-based prioritization into PPO, the algorithm evaluates the similarity between the current state and all stored transitions, assigning a higher probability of sampling to transitions that exhibit greater similarity. By focusing on transitions with similar states to the current one, we ensure that the agent learns from experiences that are more relevant to its current situation. Prioritizing similar transitions leads to more efficient updates since transitions that resemble the current state are more likely to provide useful information for decision-making.

C. The Loss Function

We modify the loss function of the value network in (5) as follows:

$$L^{\text{value}}(\theta) = \omega_i(\mathcal{L}_{\text{1-step}} + \lambda_1 \mathcal{L}_{\text{n-step}} + \lambda_2 \mathcal{L}_{\text{reg}}), \tag{8}$$

This updated loss function incorporates different components, including 1-step and n-step Temporal Difference (TD) losses, and an L2 norm regularization term. The parameters λ_1 and λ_2 control the relative contributions of each term to the overall loss function. By integrating these elements, the value network can effectively learn from immediate and long-term rewards, while avoiding overfitting.

The 1-step TD error, denoted as $\mathcal{L}_{\text{1-step}}$, is defined as:

$$\mathcal{L}_{\text{1-step}} = \left(V_{\theta'}(s) - R(t)\right)^2.$$

The n-step TD loss, represented as $\mathcal{L}_{\text{n-step}}$, is expressed as:

$$\mathcal{L}_{\text{n-step}} = \left(R(t)_{(n)} + \gamma^{\tau_{(n)}} V_{\theta'}(s'_{(n)}) - V_{\theta'}(s)\right)^2, \tag{9}$$

where $V_{\theta'}(s'_{(n)})$ refers to the state encountered after n steps, and $\tau_{(n)}$ is the cumulative transition time from s to $s'_{(n)}$. The cumulative discounted reward $R(t)_{(n)}$ is calculated as:

$$R(t)_{(n)} = \gamma^{\tau_{(1)}} r_{(1)} + \gamma^{\tau_{(2)}} r_{(2)} + \gamma^{\tau_{(3)}} r_{(3)} + \ldots + \gamma^{\tau_{(n)}} r_{(n)}$$
$$\tau_{(n)} = t_1 + t_2 + t_3 + \ldots + t_n,$$

where t_n represents the transition time from state $s'_{(n-1)}$ to $s'_{(n)}$. $r_{(n)}$ is the immediate reward at the nth step. Incorporating n-step returns ensures that the values of subsequent states are propagated back to preceding states, enhancing the initial training process.

$\mathcal{L}_{\text{reg}}$ is an L2 regularization term designed to reduce overfitting by penalizing large weights.

Table I presents the pseudocode of our proposed Enhanced Proximal Policy Optimization (PPO) algorithm.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PARAMETER SETTINGS

Notation	value
F	50
γ	0.99
M	10000
β	0.6 linearly increased to 1
α	0.4
Batch size	64
$\|\Lambda\|$	10000
l for each file type	Randomly generated from [10, 30]
i for each file type	Randomly generated from [0.1, 0.9]
z for each file type	Randomly generated from [100, 1000]
λ	0.2
η	[0, 1]

Algorithm 1: Enhanced PPO with Prioritized Replay Buffer and Attention Mechanism.

Initialize policy parameters θ and value network parameters θ'
Initialize prioritized replay buffer Λ
Initialize hyperparameters α, β, ϵ, γ, λ_1, λ_2
for each episode **do**
  Reset environment and initialize state s
  for each timestep t **do**
    Select action $a_t \sim \pi_\theta(a_t \mid s_t)$
    Execute a_t, observe reward r_t, and next state s_{t+1}
    Compute transition priority p_t using attention mechanism:
      $p_k = \text{score}(Q, K_k)$ where Q is the embedding of s_t and K_k is the embedding of s_k in Λ
    Store transition $(s_k, a_k, r_k, s_{k+1}, p_k)$ in Λ
  end for
  for each training step **do**
    Sample a minibatch of transitions $\{(s_i, a_i, r_i, s_{i+1}, p_i)\}$ from Λ
    Compute importance sampling weights $\omega_i = (\frac{1}{|\Lambda|} \cdot \frac{1}{P(i)})^\beta$
    Update policy parameters θ by maximizing clipped objective:
    $$L^{\text{PPO}}(\theta) = \min\left(\frac{\pi_\theta(a(t)|s(t))}{\pi_{\theta_{\text{old}}}(a(t)|s(t))}\hat{A}(t),\right.$$
    $$\left.\text{clip}(\frac{\pi_\theta(a(t)|s(t))}{\pi_{\theta_{\text{old}}}(a(t)|s(t))}, 1-\epsilon, 1+\epsilon)\hat{A}(t)\right)$$
    Compute value loss:
    $$L^{\text{value}}(\theta') = \mathcal{L}_{\text{1-step}} + \lambda_1\mathcal{L}_{\text{n-step}} + \lambda_2\mathcal{L}_{\text{reg}}$$
    Perform gradient descent on combined loss
  end for
end for

VI. EXPERIMENTAL SETUP AND RESULTS

This section begins with a description of the experimental setup and the tools utilized to implement our Proposed Caching Method (PCM). Next, we introduce two relevant and recent DRL algorithms, which will serve as baselines for comparison.

Our system model was simulated and the DRL agent was trained using Python 3. To streamline the development of neural networks, we employed the TensorFlow platform as described in [34].

A. Configuration and Parameters

The DRL algorithm was implemented with 2 neural networks for actor and critic, each featuring 3 layers. The weights of the network were initialized within the range of [-0.1, 0.1], while biases were set to 0.1. ReLU served as the activation function. Additional parameter details can be found in Table I.

File request probabilities follow a Zipf distribution, characterized by the parameter η where $0 < \eta \leq 1$. In this distribution, the likelihood of the fth file being requested is given by $\mathfrak{p}_f = \frac{1}{\sigma f^\eta}$ [30], with σ defined as:

$$\sigma = \sum_{f=1}^{F} \frac{1}{f^\eta}, \tag{10}$$

The value of η influences the skewness of the Zipf distribution. When η approaches 1, the likelihood of requesting the most popular file increases significantly compared to other files. Conversely, if η is close to 0, the popularity of files becomes more evenly distributed. The utility function is set to increase linearly with the importance of a file and exponentially with its freshness.

We utilize a non-linear utility function to obtain the utility of file type f at time t:

$$y_f(t) = (-e^{(h^f(t)+log(\mathfrak{c}))} + UT_{\max} + \mathfrak{c}) \times i_f, \tag{11}$$

where $y_f(t)$ is the utility of file type f. We define $\mathfrak{c}$ by adjusting $UT_{\max}$ and $UT_{\min}$ as $\mathfrak{c} = (UT_{\max} - UT_{\min})/(e-1)$ where $UT_{\max}$ and $UT_{\min}$ denote the maximum and minimum values of utilities. The non-linear utility function is more general than a linear one. The utility of file f depends on both its freshness and importance according to (11). If the freshness of file f decreases, its utility decreases. Similarly, if the importance of file f decreases, its utility also reduces. The rate of this reduction can be flexibly adjusted by the parameter Curve to meet the requirements of specific applications.

B. Baselines

In this part, we introduce the baseline methods used for evaluating the performance of the proposed DRL approach.

- *CTD:* Our DRL caching algorithm is compared against the approach presented in [17], where the authors modeled the caching problem as a discrete-time MDP and developed a DRL method to assist the edge router in deciding which files to cache based on various system states. In [17], file characteristics such as differing popularities and lifetimes were considered. However, other attributes like importance and file size were not included in their model. In our result comparisons, we refer to the approach in [17] as CTD.
- *RLTD:* Another baseline we use for comparison is from [6]. This baseline proposes a DRL-based caching scheme, also considering the freshness and limited lifetime of data files. They proposed a distributed DRL approach for a hierarchical architecture where each cache in the hierarchy has a separate DRL agent operating independently. Since

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

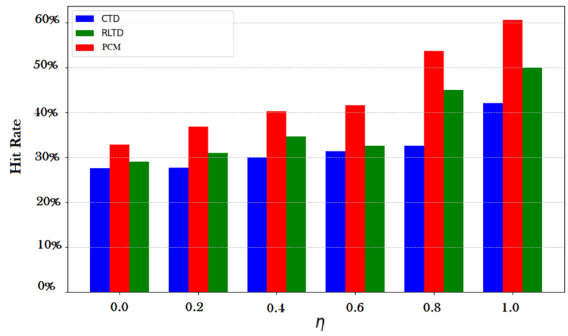


Fig. 2. Hit rate for different values for η.

our work considers a single-level cache, we adapted this baseline to our system model by implementing it within a single-level cache. We present the results based on this configuration and call it RLTD.

C. Evaluation Criteria

Our proposed method is assessed using the following performance indicators:

- *Hit Rate:* After training and policy convergence, different algorithms are tested with an additional 1000 user requests under identical conditions. Hit rate refers to the number of times a requested item is successfully retrieved from the cache. The primary objective of any caching strategy is to enhance this metric.
- *Total Utility:* When a requested file is found in the cache, its utility score is added to the cumulative utility. A higher aggregate utility signifies that the caching strategy effectively retains files with greater utility values.
- *Rate of Convergence:* An RL agent eventually stabilizes at its peak average reward. The efficiency of the algorithm is judged by how quickly it reaches this stable reward level. Rapid convergence is a key indicator of the learning algorithm's effectiveness.

D. Experimental Findings

This section begins with a comparative analysis of our simulation results against those of the CTD and RLTD methods, focusing on metrics such as total utility and cache hit rate. We also compare the performance of our enhanced PPO algorithm with the original PPO algorithm to demonstrate how our enhancements lead to faster convergence.

a) Effect of Popularity Skewness: Figs. 2 and 3 illustrate the hit rate and total utility across 1000 trials for varying values of the Zipf parameter (η). The parameter η governs the degree of skewness in the popularity distribution. A value of η near 0 indicates that file popularities are relatively uniform, meaning there is minimal disparity between the most and least popular files. Conversely, when η approaches 1, the distribution becomes highly skewed, with a few files dominating in popularity while others remain relatively obscure. This skewness means that caching the most popular files leads to a higher hit rate, as increasing η from 0 to 1 enhances the likelihood of such

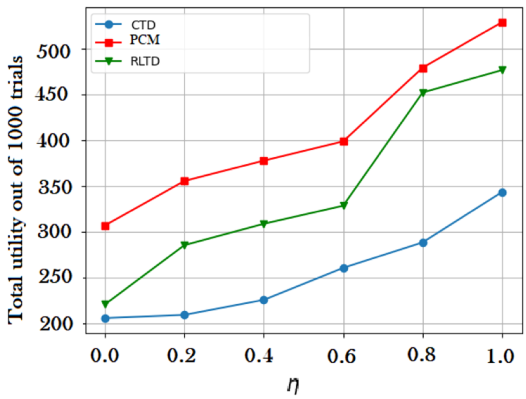


Fig. 3. Total utility for a) different values for η.

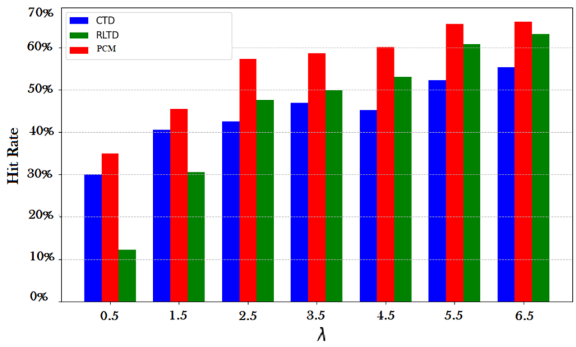


Fig. 4. Hit rate for different values for λ.

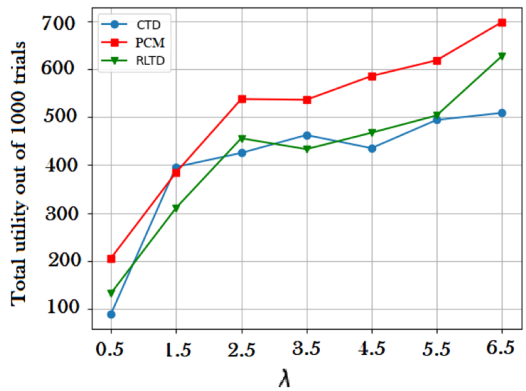


Fig. 5. Total utility for different values for λ.

files being requested. Consequently, as η rises, the total utility also increases due to the higher probability of retaining more frequently requested files, which are more likely to be up-to-date.

PCM outperforms both CTD and RLTD in terms of hit rate and total utility across all values of η, ranging from 0 to 1. This performance gap is due to PCM's ability to consider a wider range of file features, such as popularity, lifetime, importance, and size, which allows it to make more informed caching decisions. As the skewness η increases, indicating a more concentrated demand for popular files, PCM optimizes cache usage by prioritizing high-utility files based on their combined features, not just popularity and lifetime like CTD and RLTD.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

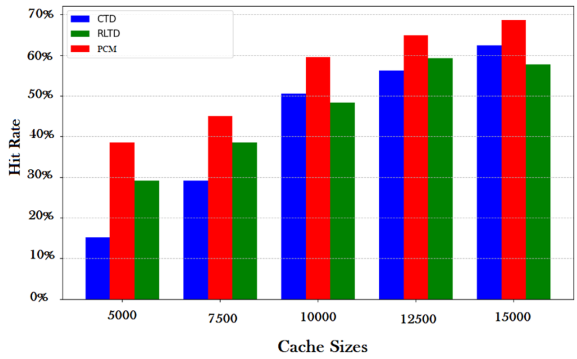


Fig. 6. Hit rate for different cache sizes.

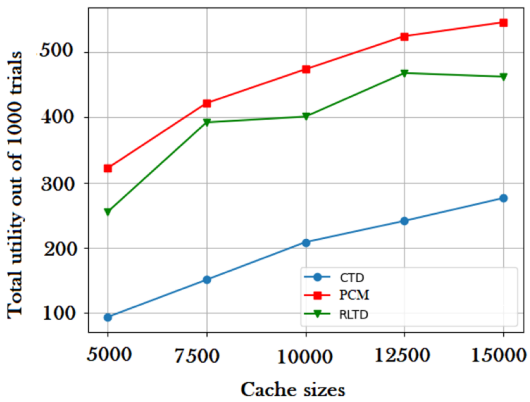


Fig. 7. Total utility for different cache sizes.

b) Effect of Request Rates: Figs. 4 and 5 depict the hit rate and total utility for varying request rates, denoted by λ, across 1000 trials. A lower λ corresponds to longer intervals between consecutive requests, which is associated with reduced hit rate. This is because longer interarrival times increase the likelihood of cached files expiring before reuse due to their finite lifetimes. Conversely, a higher λ means requests are more frequent, allowing the cache to fulfill more requests before files expire. This results in better performance regarding both hit rate and total utility. Our proposed method demonstrates superior performance compared to existing approaches for different values of λ.

c) Effect of Cache Size: Increasing the cache size enhances the system's capacity to store more files, which generally leads to a higher number of requests being served directly from the cache rather than fetching them from the data center. This increased capacity results in improved hit rate with larger caches. As illustrated in Fig. 6, our proposed approach consistently performs better than the benchmark caching strategies across various cache sizes. Additionally, as shown in Fig. 7, our method also surpasses the benchmark methods in terms of total utility.

d) Comparison of MDP and SMDP Approaches: To showcase the benefits of using SMDP, we experiment with an alternative version of our caching algorithm, where the problem is framed using a discrete-time MDP instead of SMDP. In the discrete-time MDP setup, decisions are made at the beginning of each time interval, causing the agent to delay actions until the next time slot arrives, even if there are pending requests. Table II shows the hit rate for the MDP and SMDP models. Since the agent in the MDP framework waits for a new time slot to make a decision, the files already in the cache continue to age, and they may even expire before being accessed. In contrast, SMDP allows the agent to make decisions as soon as a request arrives, optimizing the time that files remain cached for serving requests. As a result, the hit rate improves because cached files are utilized more efficiently. This increase in hit rate is especially significant when the request rate is higher, as more frequent requests with smaller interarrival times benefit from the SMDP's ability to respond immediately, reducing the likelihood of expired or outdated cached files.

TABLE II
HIT RATE FOR MDP AND SMDP UNDER DIFFERENT REQUEST RATES

	$\lambda = 5$	$\lambda = 1.66$	$\lambda = 1.0$
MDP	405	431	399
SMDP	610	526	443

Through extensive experimentation, we demonstrate that PCM significantly outperforms both CTD and RLTD in terms of cache hit rate and total utility under various scenarios. Additionally, we find that RLTD performs better than CTD. In the following sections, we explain the key reasons for these performance differences.

The superior performance of PCM can be attributed to two major contributions. First, PCM incorporates a comprehensive set of file features in its decision-making process, including file popularity, lifetime, importance, and size. In contrast, both CTD and RLTD limit their scope to only two features: file lifetime and popularity. By considering a richer set of features, PCM is better equipped to optimize caching decisions based on the specific characteristics of each file. This allows PCM to prioritize files not only based on their popularity but also their importance and size, leading to more efficient use of the cache and improved hit rates.

Second, PCM is formulated using SMDP, which enables decision-making at any moment when a request arrives at the edge router. This flexibility allows PCM to immediately respond to incoming requests and take action as needed, maximizing the use of cached files. On the other hand, both CTD and RLTD are based on a standard MDP, where decisions are only made at the beginning of fixed time intervals. As a result, when requests arrive between time intervals, the system must wait until the next interval to take action, which can cause delays in responding to requests and lead to lower hit rates. This limitation in MDP-based approaches explains why PCM, with its continuous decision-making capability, performs better, particularly in environments with high request rates and variable interarrival times.

The advantage of RLTD over CTD lies in their respective reward functions. CTD's reward function provides instant rewards solely based on the most recent requested file, which limits the feedback to the immediate outcome of the agent's actions. In contrast, both RLTD and PCM use a more sophisticated reward function that takes into account the history of cached files. By considering a delayed reward system, these approaches better

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

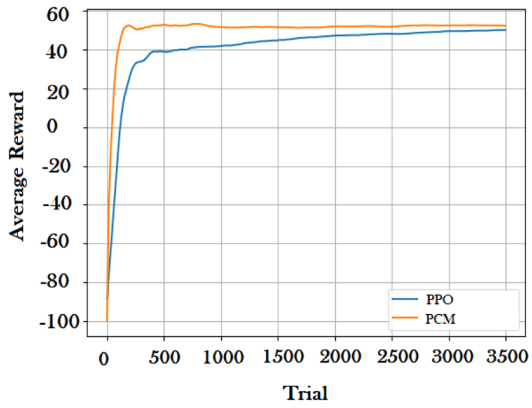


Fig. 8. PPO enhancement.

reflect the cumulative impact of previous caching decisions. This delayed feedback gives a more accurate evaluation of the agent's long-term performance, rather than focusing only on short-term gains from responding to the most recent request. Consequently, RLTD and PCM can optimize caching policies more effectively over time, leading to better overall results compared to CTD.

e) PPO Enhancement: In our simulations, we observed a significant improvement in convergence speed with the enhanced PPO algorithm that incorporates experience prioritization using an attention mechanism in the replay buffer. The results are illustrated in Fig. 8, which compares the average reward over time during the training process for the standard PPO and PCM. The plot clearly shows that PCM converges much faster, achieving higher average rewards earlier in the training process compared to the baseline PPO. Particularly, the enhanced PPO algorithm reaches its stable average reward faster than the standard PPO. For more clarity, the only difference between the PCM and the standard PPO algorithm is the use of the attention mechanism in PCM to prioritize transitions within the replay buffer. The results clearly show that PCM achieves better learning efficiency and policy performance compared to PPO, validating the impact of the attention mechanism.

This comparison effectively highlights the contribution of the proposed mechanism to the overall performance of PCM.

The rapid convergence of the enhanced PPO can be attributed to the effective prioritization of experiences in the replay buffer. By leveraging the attention mechanism, the algorithm focuses on more relevant experiences, which accelerates the learning process and improves overall performance. This prioritization ensures that the agent learns from the most impactful transitions more frequently, facilitating a more efficient exploration of the state space. Consequently, this leads to quicker adaptation and refinement of the policy, thereby achieving superior performance in less time.

VII. Conclusion

In conclusion, our research presents a novel approach to caching challenges by leveraging an SMDP model to handle real-world scenarios, where file requests arrive randomly at the edge router. Our proposed PPO-based caching method integrates a wide range of file attributes, such as popularity, lifetime, size, and importance. Simulation results highlight its superior performance compared to existing DRL-based methods, demonstrating improved efficiency. Additionally, we have enhanced the PPO algorithm by incorporating an attention mechanism to prioritize transitions in the replay buffer, leading to accelerated convergence and further performance improvements.

References

[1] Y. Liu, J. Jia, J. Cai, and T. Huang, "Deep reinforcement learning for reactive content caching with predicted content popularity in three-tier wireless networks," *IEEE Trans. Netw. Serv. Manage.*, vol. 20, no. 1, pp. 486–501, Mar. 2023.

[2] I. U. Din, S. Hassan, M. K. Khan, M. Guizani, O. Ghazali, and A. Habbal, "Caching in information-centric networking: Strategies, challenges, and future research directions," *IEEE Commun. Surveys Tut.*, vol. 20, no. 2, pp. 1443–1474, Apr.–Jun. 2018.

[3] S. Borst, V. Gupta, and A. Walid, "Distributed caching algorithms for content distribution networks," in *Proc. IEEE INFOCOM*, 2010, pp. 1–9.

[4] J. Shuja, K. Bilal, W. Alasmary, H. Sinky, and E. Alanazi, "Applying machine learning techniques for caching in next-generation edge networks: A comprehensive survey," *J. Netw. Comput. Appl.*, vol. 181, 2021, Art. no. 103005.

[5] H. Ahlehagh and S. Dey, "Video-aware scheduling and caching in the radio access network," *IEEE/ACM Trans. Netw.*, vol. 22, no. 5, pp. 1444–1462, Oct. 2014.

[6] H. Wu, A. Nasehzadeh, and P. Wang, "A deep reinforcement learning-based caching strategy for IoT networks with transient data," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13310–13319, Dec. 2022.

[7] S. Ahangary, H. Chitsaz, M. J. Sobouti, A. H. Mohajerzadeh, M. H. Yaghmaee, and H. Ahmadi, "Reactive caching of viral content in 5G networks," in *Proc. 3rd Int. Conf. Adv. Commun. Technol. Netw.*, 2020, pp. 1–7.

[8] Z. Zhang, Y. Yang, M. Hua, C. Li, Y. Huang, and L. Yang, "Proactive caching for vehicular multi-view 3 D video streaming via deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 18, no. 5, pp. 2693–2706, May 2019.

[9] W. Jiang, G. Feng, S. Qin, T. S. P. Yum, and G. Cao, "Multi-agent reinforcement learning for efficient content caching in mobile D2D networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 3, pp. 1610–1622, Mar. 2019.

[10] J. Rao, H. Feng, C. Yang, Z. Chen, and B. Xia, "Optimal caching placement for D2D assisted wireless caching networks," in *Proc. IEEE Int. Conf. Commun.*, 2016, pp. 1–6.

[11] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, Inc., 2014.

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT press, 2018.

[13] P. Wu, J. Li, L. Shi, M. Ding, K. Cai, and F. Yang, "Dynamic content update for wireless edge caching via deep reinforcement learning," *IEEE Commun. Lett.*, vol. 23, no. 10, pp. 1773–1777, Oct. 2019.

[14] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs : A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, no. 1–2, pp. 181–211, 1999.

[15] F. Niknia, P. Wang, A. Agarwal, and Z. Wang, "Edge caching based on deep reinforcement learning," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2023, pp. 1–6.

[16] N. Zhang, W. Wang, P. Zhou, and A. Huang, "Delay-optimal edge caching with imperfect content fetching via stochastic learning," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 1, pp. 338–352, Mar. 2022.

[17] H. Zhu, Y. Cao, X. Wei, W. Wang, T. Jiang, and S. Jin, "Caching transient data for Internet of Things: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 2074–2083, Apr. 2019.

[18] A. Nasehzadeh and P. Wang, "A deep reinforcement learning-based caching strategy for Internet of Things," in *Proc. IEEE/CIC Int. Conf. Commun.*, 2020, pp. 969–974.

[19] J. Yao and N. Ansari, "Caching in dynamic IoT networks by deep reinforcement learning," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3268–3275, Mar. 2021.

[20] C. Zhong, M. C. Gursoy, and S. Velipasalar, "Deep reinforcement learning-based edge caching in wireless networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 1, pp. 48–61, Mar. 2020.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.

[21] C. Sun, X. Li, J. Wen, X. Wang, Z. Han, and V. C. M. Leung, "Federated deep reinforcement learning for recommendation-enabled edge caching in mobile edge-cloud computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 3, pp. 690–705, Mar. 2023.
[22] X. Huang, Z. Chen, Q. Chen, and J. Zhang, "Federated learning based QoS-aware caching decisions in fog-enabled Internet of Things networks," *Digit. Commun. Netw.*, vol. 9, no. 2, pp. 580–589, 2023.
[23] D. Gupta, S. Rani, B. Tiwari, and T. Gadekallu, "An edge communication based probabilistic caching for transient content distribution in vehicular networks," *Sci. Rep.*, vol. 13, 2023, Art. no. 3614.
[24] X. Wei, J. Liu, Y. Wang, C. Tang, and Y. Hu, "Wireless edge caching based on content similarity in dynamic environments," *J. Syst. Archit.*, vol. 115, 2021, Art. no. 102000.
[25] X. Zhou et al., "Edge computation offloading with content caching in 6 G-enabled IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2733–2747, Mar. 2024.
[26] J. Gao, S. Zhang, L. Zhao, and X. Shen, "The design of dynamic probabilistic caching with time-varying content popularity," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1672–1684, Apr. 2021.
[27] L. Zhao, H. Li, N. Lin, M. Lin, C. Fan, and J. Shi, "Intelligent content caching strategy in autonomous driving toward 6 G," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 9786–9796, Jul. 2022.
[28] S. Li, J. Xu, M. van der Schaar, and W. Li, "Popularity-driven content caching," in *Proc. IEEE INFOCOM 35th Annu. IEEE Int. Conf. Comput. Commun.*, 2016, pp. 1–9.
[29] S. Vural, N. Wang, P. Navaratnam, and R. Tafazolli, "Caching transient data in internet content routers," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 1048–1061, Apr. 2017.
[30] H. Gomaa, G. G. Messier, C. Williamson, and R. Davies, "Estimating instantaneous cache hit ratio using markov chain analysis," *IEEE/ACM Trans. Netw.*, vol. 21, no. 5, pp. 1472–1483, Oct. 2013.
[31] F. Niknia, V. Hakami, and K. Rezaee, "An SMDP-based approach to thermal-aware task scheduling in NoC-based MPSoC platforms," *J. Parallel Distrib. Comput.*, vol. 165, pp. 79–106, 2022.
[32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[33] P. Poupart, "Semi-Markov decision processes (SMDPs)," University of Waterloo, Accessed: Mar. 27, 2025. [Online]. Available: https://cs.uwaterloo.ca/~ppoupart/teaching/cs885-spring18/slides/cs885-lecture15c.pdf
[34] T. Developers, "Tensorflow," *Zenodo*, 2021. Accessed: Mar. 27, 2025. [Online]. Available: https://ui.adsabs.harvard.edu/abs/2021zndo...4758419D/abstract



Farnaz Niknia received the B.Sc. degree from the University of Tabriz, Tabriz, Iran, and M.Sc. degree from the Iran University of Science and Technology, Tehran, Iran. She is currently working toward the Ph.D. degree in electrical engineering and computer science with York University, Toronto, Canada. In both programs, she was recognized as an exceptional talent. Her research interests include reinforcement learning, edge computing, and learning in dynamic environments.



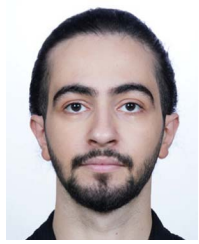
Ping Wang is currently a Professor with the Department of Electrical Engineering and Computer Science, York University, Toronto, Canada, and a Tier 2 York Research Chair. She was with Nanyang Technological University, Singapore, from 2008 to 2018. Her recent research interests focuses on integrating Artificial Intelligence (AI) techniques into communications networks. Her scholarly works have been widely disseminated through top-ranked IEEE journals/conferences. She is also the Associate Editor-in-Chief for IEEE COMMUNICATIONS SURVEYS AND TUTORIALS and an Editor for several reputed journals, including IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. She is a Distinguished Lecturer of the IEEE Vehicular Technology Society. She is also the Chair of the Education Committee of IEEE VTS.



Zixu Wang received the bachelor's degree from the South China University of Technology, Guangzhou, China, (SCUT) in 2023, and is currently working toward the Ph.D. degree with the Hong Kong University of Science and Technology, Hong Kong, (HKUST). During his study in SCUT, he was awarded the National Scholarship in 2020 and 2021. He was admitted to HKUST through the Ph.D. Early Admission Scheme which aimed at top 1% students. He was also awarded the Red Bird Scholarship with HKUST,. His current research interests include neural engineering and brain machine interfaces.



Aakash Agarwal received the bachelor's degree of Technology in electronics and communication engineering from the International Institute of Information Technology, Bangalore, India. He is currently working toward the master's degree of Science in electrical and systems engineering with the University of Pennsylvania, USA. His research interests include machine learning, deep learning, optimization, and neural network compression.



Adib S. Rezaei received the bachelor's degree from the University of Tehran, Tehran, Iran, in 2024. He is currently working toward the Ph.D. degree in computer science with Virginia Tech, Blacksburg, VA, USA. His research interests include GPU scheduling and cloud computing.

Authorized licensed use limited to: York University. Downloaded on November 03,2026 at 23:52:25 UTC from IEEE Xplore. Restrictions apply.